Mirror Networking 초심자안내서

> Unity 멀티플레이어게임개발을위한고수준네트워킹라이브러리

목차

1. Mirror Networking 이란?
2. 설치및설정
3. 핵심개념이해하기
4. 기본컴포넌트
5. 첫멀티플레이어게임만들기
6. 네트워크동기화
7. 원격작업 (Commands & RPCs)
8. FAQ 및문제해결
9. 학습리소스

---

1. Mirror Networking 이란?

개요

Mirror 는 Unity 에서멀티플레이어게임을쉽게개발할수있도록도와주는 **고수준네트워킹라이브러리**입니다. 사용하기쉽고 성공확률을높이는것에최적화되어있습니다.

주요특징

특징	설명
🎮 **고수준 API**	복잡한네트워크구현세부사항을걱정할필요없이게임로직에집중
🔌 **다양한트랜스포트지원**	12 개이상의저수준트랜스포트와호환
📦 **풍부한스크립트템플릿**	학습과코딩을쉽게만드는템플릿제공
🎯 **다양한컴포넌트**	12 개이상의내장컴포넌트
🌍 **관심영역관리**	5 가지방식의 Interest Management 제공
🏠 **호스트-클이언트모델**	전용서버없이도플레이어가호스트가될수있음

Mirror 의구조

```
┌──────────────────────────────────┐
│          게임 로직 (Game Logic)          │
├──────────────────────────────────┤
│       Mirror 고수준 API (HLAPI)          │
├──────────────────────────────────┤
│       저수준 트랜스포트 (Transport)       │
├──────────────────────────────────┤
│       물리적 네트워크 연결 (TCP/UDP)       │
└──────────────────────────────────┘
```

---

2. 설치및설정

요구사항

- **Unity 버전**: 2020 LTS 또는 2021 LTS 권장
- **플랫폼**: Windows, Mac, Linux, iOS, Android 등

설치방법

방법 1: Unity Asset Store

1. Unity Asset Store에서 Mirror 다운로드
2. Unity 프로젝트에가져오기

방법 2: GitHub (Package Manager)

```
https://github.com/MirrorNetworking/Mirror.git
```

기본설정확인

설치후다음이자동으로추가됩니다:

- `Assets/Mirror/` 폴더에모든파일
- 스크립트템플릿 (`Assets > Create > Mirror` 메뉴)

---

3. 핵심개념이해하기

3.1 서버-클리언트모델

```
┌──────────────────────────────────────────────────────┐
│                      네트워크 구조                      │
├──────────────────────────────────────────────────────┤
│                                                      │
│    ┌──────────┐      ┌──────────┐      ┌──────────┐   │
│    │ 클라이언트 1 │◄────►│   서버   │◄────►│ 클라이언트 2 │   │
│    │ (플레이어 1)│      │ (게임 상태 │      │ (플레이어 2)│   │
│    └──────────┘      │  관리자)  │      └──────────┘   │
│                      └──────────┘                      │
│                           │                            │
│                      ┌──────────┐                      │
│                      │ 클라이언트 3 │                      │
│                      │ (플레이어 3)│                      │
│                      └──────────┘                      │
│                                                      │
└──────────────────────────────────────────────────────┘
```

3.2 주요용어

용어	설명
Server (서버)	모든플레이어가연결되는게임인스턴스. 게임상태를관리하고점수를추적
Client (클리언트)	서버에연결되는게임인스턴스. 플레이어가게임을플레이하는창
Host (호스트)	서버이면서동시에클라이언트인인스턴스 (플레이어가서버역할도함)
Dedicated Server (전용서버)	플레이어없이오직서버역할만하는인스턴스

3.3 서버권한 (Server Authority)

Mirror 는 **서버권한시스템**을사용합니다:

- 모든중요한결정은서버가내림
- 클라이언트는입력만서버에전송
- 서버가게임상태를검증하고동기화

이는치팅을방지하는데매우효과적입니다!

---

4. 기본컴포넌트

4.1 필수컴포넌트개요

컴포넌트	설명	필수여부
NetworkManager	네트워크게임의전반적인관리	⭐ 필수
NetworkIdentity	네트워크상의고유 ID 부여	⭐ 필수
NetworkBehaviour	네트워크스크립트의기본클래스	⭐ 필수
NetworkTransform	위치/회전동기화	선택적
NetworkAnimator	애니메이션상태동기화	선택적
NetworkManagerHUD	기본 UI 제공 (개발용)	선택적

4.2 NetworkManager

네트워크게임의 “뇌” 역할을합니다.

주요기능:

- 게임시작/종료관리
- 플레이어스폰관리
- 씬관리
- 네트워크설정

설정방법:

```
1. Assets > Create > Mirror > Network Manager 선택
2. 씬에 빈 게임 오브젝트 생성 (이름: "NetworkManager")
3. 생성된 Network Manager 컴포넌트 추가
4. NetworkManagerHUD 컴포넌트도 추가 (개발 테스트용 UI)
```

4.3 NetworkIdentity

모든네트워크오브젝트에필요한 “신분증” 입니다.

특징:

- 네트워크상에서오브젝트를고유하게식별
- 서버/클리언트권한설정가능
- 자동으로네트워크 ID 할당

Inspector 설정:

```
Server Only (서버에서만 존재)
Client Authority (클리언트가 권한 가짐)
```

4.4 NetworkBehaviour

네트워크기능이필요한모든스크립트의기본클래스입니다.

```
using UnityEngine;
using Mirror;

// MonoBehaviour 대신 NetworkBehaviour 사용
public class MyNetworkScript : NetworkBehaviour
{
    void Update()
    {
        // 네트워크 관련 기능 사용 가능
    }
}
```

주요속성:

- `isServer`: 현재인스턴스가서버인지확인
- `isClient`: 현재인스턴스가클라이언트인지확인
- `isLocalPlayer`: 이오브젝트가로컬플레이어인지확인
- `hasAuthority`: 이오브젝트에대한권한이있는지확인

---

5. 첫멀티플레이어게임만들기

5.1 단계별튜토리얼

단계 1: Network Manager 설정

```
1. 씬에 빈 게임 오브젝트 생성 → 이름을 "NetworkManager" 로 변경
2. Assets > Create > Mirror > Network Manager 선택하여 스크립트 생성
3. NetworkManager 컴포넌트를 게임 오브젝트에 추가
4. NetworkManagerHUD 컴포넌트도 추가
```

단계 2: 플레이어프리팹만들기

```
1. 기존 플레이어 오브젝트를 프리팹으로 변환 (또는 새로 생성)
2. 프리팹에 NetworkIdentity 컴포넌트 추가
3. 프리팹에 NetworkTransform 컴포넌트 추가
   - "Client Authority" 체크박스 선택
4. NetworkManager 의 Spawn Info > Player Prefab 에 프리팹 할당
5. 씬에서 원본 플레이어 오브젝트 제거 (NetworkManager 가 자동 스폰)
```

단계 3: 플레이어이동스크립트

```
using UnityEngine;
using Mirror;

public class PlayerController : NetworkBehaviour
{
    public float moveSpeed = 5f;

    void Update()
    {
        // 로컬 플레이어만 입력 처리
        if (!isLocalPlayer)
            return;

        // 이동 입력
        float horizontal = Input.GetAxis("Horizontal");
        float vertical = Input.GetAxis("Vertical");

        Vector3 movement = new Vector3(horizontal, 0, vertical) * moveSpeed * Time.deltaTime;
        transform.Translate(movement);
    }

    // 로컬 플레이어가 시작될 때 호출
    public override void OnStartLocalPlayer()
    {
        // 메인 칩라를 플레이어에 연결
        Camera.main.transform.SetParent(transform);
        Camera.main.transform.localPosition = new Vector3(0, 2, -5);
    }
}
```

단계 4: 게임테스트

```
1. Unity 에디터에서 Play 클릭
2. NetworkManagerHUD 에서 "Host (Server + Client)" 클릭
3. File > Build And Run 으로 빌드 실행
4. 빌드된 게임에서 "Client" 선택하고 IP 입력 (localhost 테스트 시 "localhost")
5. 두 클라이언트가 서로를 볼 수 있는지 확인!
```

Generated by Kimi.ai

6. 네트워크동기화

6.1 SyncVar (동기화변수)

서버에서모든클라이언트로자동동기화되는변수입니다.

```
using UnityEngine;
using Mirror;

public class PlayerHealth : NetworkBehaviour
{
    // 서버에서 변경되면 모든 클라이언트에 자동 동기화
    [SyncVar]
    public int health = 100;

    [SyncVar]
    public string playerName = "Player";

    // Hook 함수로 변경 감지
    [SyncVar(hook = nameof(OnScoreChanged))]
    public int score = 0;

    void OnScoreChanged(int oldScore, int newScore)
    {
        Debug.Log($" 점수 변경: {oldScore} → {newScore}");
        // UI 업데이트 등
    }

    // 서버에서만 호출 가능
    [Server]
    public void TakeDamage(int damage)
    {
        health -= damage;
        if (health <= 0)
        {
            // 사망 처리
        }
    }
}
```

6.2 동기화가능한데이터타입

기본타입 (자동지원):

- int, float, bool, string
- Vector2, Vector3, Vector4
- Quaternion, Color
- 배열 (int[], string[] 등)

커스텀구조체:

```
public struct PlayerData
```

```
{
    public string name;
    public int level;
    public Vector3 position;
}

// Mirror 가 자동으로 직렬화 함수 생성
[SyncVar]
public PlayerData playerData;
```

6.3 NetworkTransform 상세

속성	설명	권장설정
Sync Direction	동기화방향	Client to Server (클리언트권한시)
Sync Position	위치동기화	체크
Sync Rotation	회전동기화	필요시체크
Sync Scale	크기동기화	필요시체크
Position/Rotation/Scale Sensitivity	동기화민감도	0.01 정도

---

7. 원격작업 (Commands & RPCs)

7.1 Command (클리언트 → 서버)

클리언트가서버에작업을요청할때사용합니다.

```
using UnityEngine;
using Mirror;

public class PlayerCombat : NetworkBehaviour
{
    [SyncVar]
    public int ammo = 30;

    void Update()
    {
        if (!isLocalPlayer)
            return;

        if (Input.GetButtonDown("Fire1"))
        {
            // 서버에 발사 요청
            CmdFire();
        }
    }

    // Command: 클라이언트가 서버에 호출
    [Command]
```

```
    void CmdFire()
    {
        if (ammo > 0)
        {
            ammo--;

            // 서버에서 처리 후 결과를 모든 클라이언트에 전파
            RpcOnFire();

            // 물리 계산 등 서버 권한 작업 수행
            // Raycast, 데미지 계산 등
        }
    }
}
```

Command 규칙:

- 함수명은 `Cmd` 로시작
- `[Command]` 어트리뷰트필요
- 서버에서만실행됨
- 클라이언트에서호출가능

7.2 ClientRpc (서버 → 모든클라이언트)

서버가모든클라이언트에작업을알릴때사용합니다.

```
[ClientRpc]
void RpcOnFire()
{
    // 모든 클라이언트에서 실행
    // 효과음 재생, 파티클 효과 등
    GetComponent<AudioSource>().PlayOneShot(fireSound);
    muzzleFlash.Play();
}
```

ClientRpc 규칙:

- 함수명은 `Rpc` 로시작
- `[ClientRpc]` 어트리뷰트필요
- 서버에서만호출가능
- 모든클라이언트에서실행

7.3 TargetRpc (서버 → 특정클라이언트)

특정클라이언트만작업을수행하도록할때사용합니다.

```
[TargetRpc]
void TargetShowDamage(NetworkConnection target, int damage)
{
    // 지정된 클라이언트에서만 실행
```

```
    damageText.text = $"-{damage}";
    damageAnimator.Play("ShowDamage");
}

// 사용 예
[Command]
void CmdTakeDamage(int damage)
{
    health -= damage;

    // 피해를 입은 플레이어에게만 메시지
    TargetShowDamage(connectionToClient, damage);
}
```

7.4 흐름도

```
┌──────────────────────────────────────────────────────────┐
│                     원격 작업 흐름도                        │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  클라이언트 A          서버            클라이언트 B, C, D...    │
│      │                 │                    │                │
│      │  [Command]      │                    │                │
│      │  CmdFire() ────►│                    │                │
│      │                 │                    │                │
│      │                 │  [ClientRpc]       │                │
│      │                 │  RpcOnFire() ────►├──────────────►│
│      │                 │                    │                │
│      │  [TargetRpc]    │                    │                │
│      │◄─── TargetShowDamage()               │                │
│      │  (특정 클라이언트만)                   │                │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

---

8. FAQ 및문제해결

자주묻는질문

Q: 같은컴퓨터에서테스트하려면?

```
Network Address 를 "localhost" 로 설정
또는 "127.0.0.1" 사용
```

Q: 같은네트워크 (LAN) 에서테스트하려면?

```
1. 호스트 컴퓨터의 LAN IP 확인 (Windows: ipconfig)
2. 클라이언트의 Network Address 를 해당 IP 로 설정
   예: "192.168.1.100", "10.0.0.5"
```

Q: 인터넷을통해연결하려면?

```
- 포트 포워딩 설정 필요
- 또는 릴리스 서비스 (Edgegap, AWS 등) 사용
- 상세 내용은 Hosting 가이드 참조
```

Q: 플레이어수를어떻게알수있나요?

```
// 현재 연결된 플레이어 수
int playerCount = NetworkServer.connections.Count;

// 또는 NetworkManager 에서
int numPlayers = NetworkManager.singleton.numPlayers;
```

일반적인문제해결

문제	원인	해결방법
클라이언트가연결되지않음	IP/포트설정오류	올바른 IP 주소확인
플레이어가보이지않음	Player Prefab 미설정	NetworkManager 에 Player Prefab 할당
이동이동기화되지않음	NetworkTransform 누락	NetworkTransform 컴포넌트추가
입력이작동하지않음	isLocalPlayer 체크누락	`if (!isLocalPlayer) return;` 추가
Command 가작동하지않음	서버권한문제	`[Command]` 어트리뷰트확인

---

9. 학습리소스

공식리소스

리소스	링크	설명
공식문서	mirror-networking.gitbook.io	상세문서
GitHub	github.com/MirrorNetworking	소스코드
Discord	discord.gg/2BvnM4R	실시간지원
API Reference	API 문서	API 상세

예제프로젝트

Mirror 에는다음과같은예제가포함되어있습니다:

예제	설명
Basic	가장기본적인네트워크게임
Pong	클래식퐁게임
Tanks	탱크전투게임
Billiards	당구게임 (예측포함)
Additive Scenes	동적씬로딩
Room	로비/방시스템
Chat	채팅시스템예제

Table 7 – continued

예제	설명
Lag Compensation	지연보정예제

비디오튜토리얼

커뮤니티에서제작한 비디오튜토리얼 목록을확인하세요.

요약체크리스트

첫멀티플레이어게임만들기

- Mirror 설치
- NetworkManager 생성및설정
- NetworkManagerHUD 추가
- 플레이어프리팹에 NetworkIdentity 추가
- 플레이어프리팹에 NetworkTransform 추가
- NetworkManager 에 Player Prefab 할당
- 플레이어스크립트를 NetworkBehaviour 로변경
- `isLocalPlayer` 체크로로컬입력처리
- 필요시 SyncVar 로상태동기화
- 필요시 Command/ClientRpc 로원격작업

마무리

Mirror Networking 은 Unity 에서멀티플레이어게임을개발하는가장쉬운방법중하나입니다. 서버권한모델을기반으로하여안전하고확장가능한네트워크게임을만들수있습니다.

다음단계:

1. Basic 예제를통해기본개념익히기
2. 자신만의간단한게임만들어보기
3. Interest Management 로대규모게임최적화학습
4. Lag Compensation 으로반응성있는게임만들기

행운을빕니다! 🎮

문서버전: 2024 년기준 Mirror Networking 원본문서: Mirror Networking Docs